

Spa Wellness

Project Report — Academic Session 2025–26

Project Title:	Spa Wellness Web Platform
Session:	2025 – 2026
Technology Stack:	MERN (MongoDB, Express, React, Node.js)
Team Members:	Member 1 Member 2 Member 3
Supervisor:	Project Supervisor Name
Institution:	Institution / Department Name
Submission Date:	April 2026

Submitted in partial fulfilment of the course requirements

Table of Contents

- 1. Abstract
- 2. Introduction
 - 2.1 Background & Motivation
 - 2.2 Problem Statement
 - 2.3 Objectives
 - 2.4 Scope
- 3. System Design & Architecture
 - 3.1 System Overview
 - 3.2 Architecture Diagram (Description)
 - 3.3 Database Design
 - 3.4 API Design
- 4. Implementation
 - 4.1 Development Environment
 - 4.2 Frontend Implementation
 - 4.3 Backend Implementation
 - 4.4 Authentication & Security
 - 4.5 Key Code Highlights
- 5. Testing & Results
 - 5.1 Testing Strategy
 - 5.2 Test Cases
 - 5.3 Results & Observations
- 6. Challenges & Learnings
- 7. Conclusion & Future Scope
 - 7.1 Conclusion
 - 7.2 Future Enhancements
- 8. References

1. Abstract

This report documents the design, development, and deployment of **Spa Wellness**, a full-stack web application built for the academic session 2025–26. The platform enables customers to browse spa services, book appointments, and track booking statuses online, while providing administrators with a centralised dashboard to manage and approve requests.

The application is developed using the **MERN stack** — MongoDB, Express.js, React.js, and Node.js — with Tailwind CSS for responsive styling and JWT-based authentication for security. The system addresses the operational inefficiencies of manual appointment scheduling in wellness businesses and replaces them with a seamless, role-based digital workflow.

2. Introduction

2.1 Background & Motivation

The wellness and spa industry is one of the fastest-growing service sectors globally. Despite this growth, many independent wellness centers continue to rely on phone calls, walk-ins, or paper registers for appointment management. This leads to scheduling conflicts, missed bookings, and a poor customer experience.

The Spa Wellness project was conceived to digitise this process — giving clients the convenience of online booking and giving operators real-time visibility over their schedule, all through a modern, aesthetically aligned web interface.

2.2 Problem Statement

Traditional spa management systems suffer from several limitations: manual appointment logs are error-prone, there is no real-time visibility into booking status for customers, and staff have no centralised tool to approve or reject requests efficiently. There is a clear need for a lightweight, web-based platform that bridges this gap.

2.3 Objectives

- Provide an intuitive service catalogue for customers to explore available treatments.
- Enable authenticated users to book appointments by selecting a service, date, and time.
- Implement role-based access control (Customer vs. Admin).
- Allow administrators to approve or reject booking requests from a central dashboard.
- Ensure data security through password hashing and JWT-based authentication.
- Deliver a responsive, wellness-themed UI using React and Tailwind CSS.

2.4 Scope

The scope of this project covers the complete development of the Spa Wellness web application, from requirements gathering through design, implementation, and testing. The application supports three user roles — Guest, Customer, and Admin — and encompasses user registration, login, service browsing,

appointment booking, status tracking, and admin-level booking management. Mobile responsiveness and basic feedback mechanisms are included within scope. Third-party payment integration and a native mobile app are considered out of scope for this session.

3. System Design & Architecture

3.1 System Overview

Spa Wellness follows a **three-tier client-server architecture**. The presentation layer is a React/Vite SPA hosted separately from the backend. The application layer is a RESTful API server built with Node.js and Express.js. The data layer is a MongoDB Atlas cluster accessed via Mongoose ODM.

3.2 Architecture Description

Layer	Technology	Responsibility
Presentation	React.js + Vite + Tailwind CSS	UI rendering, routing, user interaction
Application	Node.js + Express.js	REST API, business logic, middleware
Data	MongoDB Atlas + Mongoose	Persistent storage, schema validation
Auth	JWT + Bcrypt.js	Session management, password security

3.3 Database Design

MongoDB is used as a NoSQL document store. Three primary collections are defined:

Users Collection

Field	Type	Description
_id	ObjectId	Auto-generated primary key
name	String	Full name of the user
email	String	Unique email address
password	String	Bcrypt-hashed password
role	String	'user' (default) or 'admin'
createdAt	Date	Account creation timestamp

Services Collection

Field	Type	Description
_id	ObjectId	Auto-generated primary key
name	String	Service name (e.g., Hot Stone Massage)
description	String	Detailed service description

Field	Type	Description
duration	Number	Duration in minutes
price	Number	Price in local currency

Bookings Collection

Field	Type	Description
_id	ObjectId	Auto-generated primary key
userId	ObjectId	Reference to Users collection
serviceId	ObjectId	Reference to Services collection
date	Date	Requested appointment date
time	String	Requested time slot
status	String	'Pending' 'Accepted' 'Rejected'
createdAt	Date	Booking submission timestamp

3.4 API Design

The backend exposes a RESTful API organised into three route groups:

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register a new user
POST	/api/auth/login	Public	Login and receive JWT
GET	/api/services	Public	List all spa services
POST	/api/bookings	User	Create a new booking
GET	/api/bookings/me	User	Get current user bookings
GET	/api/admin/bookings	Admin	Get all bookings
PATCH	/api/admin/bookings/:id	Admin	Update booking status

4. Implementation

4.1 Development Environment

Tool / Technology	Version	Purpose
Node.js	20.x LTS	JavaScript runtime for backend
npm	10.x	Package management
React + Vite	18.x / 5.x	Frontend framework and build tool
Tailwind CSS	3.x	Utility-first CSS framework
Express.js	4.x	Web framework for Node.js
MongoDB Atlas	7.x	Cloud-hosted NoSQL database
Mongoose	8.x	MongoDB ODM for schema enforcement
VS Code	Latest	Primary IDE
Postman	Latest	API testing and documentation
Git / GitHub	Latest	Version control and collaboration

4.2 Frontend Implementation

The frontend is a single-page application (SPA) bootstrapped with **Vite** for fast hot-module replacement during development and optimised production bundles. **React Router v6** handles client-side routing, including protected routes that redirect unauthenticated users to the login page.

Key frontend pages and their responsibilities include:

- **Home / Landing:** Showcases the brand, featured services, and call-to-action buttons.
- **Services:** Displays all available spa treatments in a card-based catalogue.
- **Book Appointment:** A form allowing authenticated users to select a service, date, and time.
- **My Bookings:** A dashboard for users to view the status of their appointments.
- **Login / Register:** Authentication forms with client-side validation.
- **Admin Dashboard:** Admin-only view listing all bookings with Approve / Reject controls.

4.3 Backend Implementation

The backend follows the **MVC (Model-View-Controller)** pattern. Express.js routes delegate to controller functions, which interact with Mongoose models to perform CRUD operations. Middleware functions handle JWT verification and role-based access control before requests reach protected controllers.

The project directory structure is organised as follows:

```
server/
```

```
■■■ models/ # Mongoose schemas (User, Service, Booking)
■■■ controllers/ # Business logic for auth, bookings, admin
■■■ routes/ # Express route definitions
■■■ middleware/ # JWT auth & role-check middleware
■■■ config/ # Database connection (db.js)
■■■ server.js # Entry point, app setup, CORS config
```

4.4 Authentication & Security

Security is implemented at multiple layers of the stack:

- **Password Hashing:** bcryptjs with a salt factor of 10 before storing in MongoDB.
- **JWT Issuance:** A signed token is returned upon successful login, stored client-side.
- **Protected Routes:** Express middleware verifies the Bearer token on every protected endpoint.
- **Role Enforcement:** A secondary middleware checks the user role before admin-only endpoints.
- **CORS Configuration:** Origin whitelist restricts API access to the known frontend domain.
- **Input Validation:** Required field checks on both client and server prevent malformed requests.

4.5 Key Code Highlights

The following snippets illustrate the core patterns used throughout the codebase:

JWT Middleware (middleware/authMiddleware.js)

```
const jwt = require('jsonwebtoken');
const protect = async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ msg: 'No token' });
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = await User.findById(decoded.id).select('-password');
  next();
};
```

Create Booking Controller (controllers/bookingController.js)

```
const createBooking = async (req, res) => {
  const { serviceId, date, time } = req.body;
  const booking = await Booking.create({
    userId: req.user._id, serviceId, date, time,
    status: 'Pending'
  });
  res.status(201).json(booking);
};
```

};

5. Testing & Results

5.1 Testing Strategy

The application was tested using a combination of manual functional testing and API-level testing via Postman. Each feature was verified against its requirement before being merged into the main branch. Edge cases — such as duplicate email registration, expired JWT tokens, and invalid booking fields — were explicitly tested.

5.2 Test Cases

TC#	Module	Test Description	Expected	Result
TC-01	Auth	Register with valid details	Account created, 201	Pass
TC-02	Auth	Register with duplicate email	Error 400	Pass
TC-03	Auth	Login with correct credentials	JWT returned, 200	Pass
TC-04	Auth	Login with wrong password	Error 401	Pass
TC-05	Booking	Create booking (authenticated user)	Booking saved, 201	Pass
TC-06	Booking	Create booking without auth token	Error 401	Pass
TC-07	Booking	View own bookings	List returned, 200	Pass
TC-08	Admin	Access admin panel as regular user	Error 403	Pass
TC-09	Admin	Approve a pending booking	Status = Accepted	Pass
TC-10	Admin	Reject a pending booking	Status = Rejected	Pass
TC-11	UI	Booking form submit without all fields	Validation error	Pass
TC-12	UI	Responsive layout on 375px viewport	No overflow/wrapping	Pass

5.3 Results & Observations

All 12 test cases passed successfully. The application correctly enforces authentication and role-based access control. JWT expiry and CORS restrictions behave as expected. The Vite production build achieves a Lighthouse performance score above 90 on desktop. No critical bugs were found during the testing phase; minor UI refinements (spacing and colour contrast on mobile) were addressed iteratively.

6. Challenges & Learnings

JWT Token Management

Handling token expiry gracefully on the frontend — particularly silent redirects to the login page — required careful Axios interceptor configuration.

Role-Based Route Protection

Designing a reusable ProtectedRoute component in React that works for both user-only and admin-only pages without code duplication was an important architectural lesson.

MongoDB Atlas Connectivity

Initial CORS and IP-whitelist issues with MongoDB Atlas required a deeper understanding of cloud database network access configuration.

State Synchronisation

Keeping the booking status in the UI in sync with backend updates without full page reloads was addressed using React Query-style refetch patterns.

Responsive Design

Tailwind CSS's utility-first approach accelerated responsive design but required discipline in avoiding inline style overrides that conflicted with the utility classes.

7. Conclusion & Future Scope

7.1 Conclusion

The Spa Wellness project successfully delivers a functional, secure, and aesthetically consistent web application that meets all defined objectives for the 2025–26 academic session. The MERN stack proved well-suited to the domain — MongoDB's flexible schema accommodated rapid iteration, Express.js kept the API layer lean, React enabled a component-driven UI, and Node.js provided a unified JavaScript environment across the stack.

The project reinforced practical skills in full-stack development, REST API design, authentication architecture, and agile feature delivery — all of which are directly applicable to industry practice.

7.2 Future Enhancements

- **Online Payment Integration:** Integrate Razorpay or Stripe for direct booking payments.
- **Real-Time Notifications:** Use WebSockets (Socket.io) to push status updates to users instantly.
- **Email Confirmations:** Send automated booking confirmation and reminder emails via Nodemailer.
- **Service Reviews:** Allow users to rate and review services after completed appointments.

- **Analytics Dashboard:** Provide admins with booking trends, peak-hour charts, and revenue summaries.
- **Native Mobile App:** Build a React Native companion app for iOS and Android.
- **Multi-Branch Support:** Extend the data model to support multiple spa locations under one account.

8. References

- [1] MongoDB, Inc. (2024). *MongoDB Atlas Documentation*. <https://www.mongodb.com/docs/atlas/>
- [2] Meta Open Source. (2024). *React Documentation*. <https://react.dev>
- [3] OpenJS Foundation. (2024). *Node.js Documentation*. <https://nodejs.org/en/docs>
- [4] Express.js Contributors. (2024). *Express.js API Reference*. <https://expressjs.com>
- [5] Vite Contributors. (2024). *Vite Guide*. <https://vitejs.dev/guide>
- [6] Tailwind Labs. (2024). *Tailwind CSS Documentation*. <https://tailwindcss.com/docs>
- [7] Auth0 Blog. (2023). *JWT Authentication Best Practices*. <https://auth0.com/blog>
- [8] Mongoose. (2024). *Mongoose ODM Documentation*. <https://mongoosejs.com/docs>